

2026 US LLVM Developers' Meeting

27 October 2026

How to build a Python DSL for MLIR

`mlir dsl` · a core assembled through plugins

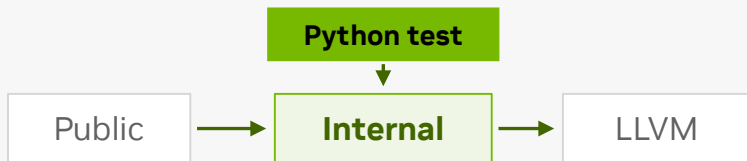
Guray Ozen

Principal Compiler Engineer, NVIDIA

Amir Tavakkoli

Why a Python DSL for MLIR?

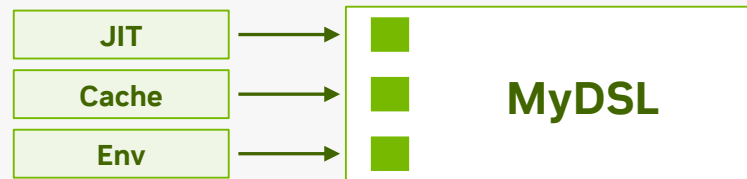
01 Test internal IR



02 Generate recipe IR



03 Reuse building blocks



04 Build a full language



NVIDIA CuTe DSL / CUTLASS Python

MLIR compilers make different choices

Different dialects.

Different passes.

Different capture rules.

How can we reuse one DSL core?

What is a plugin?

A component that supplies behavior through hooks.

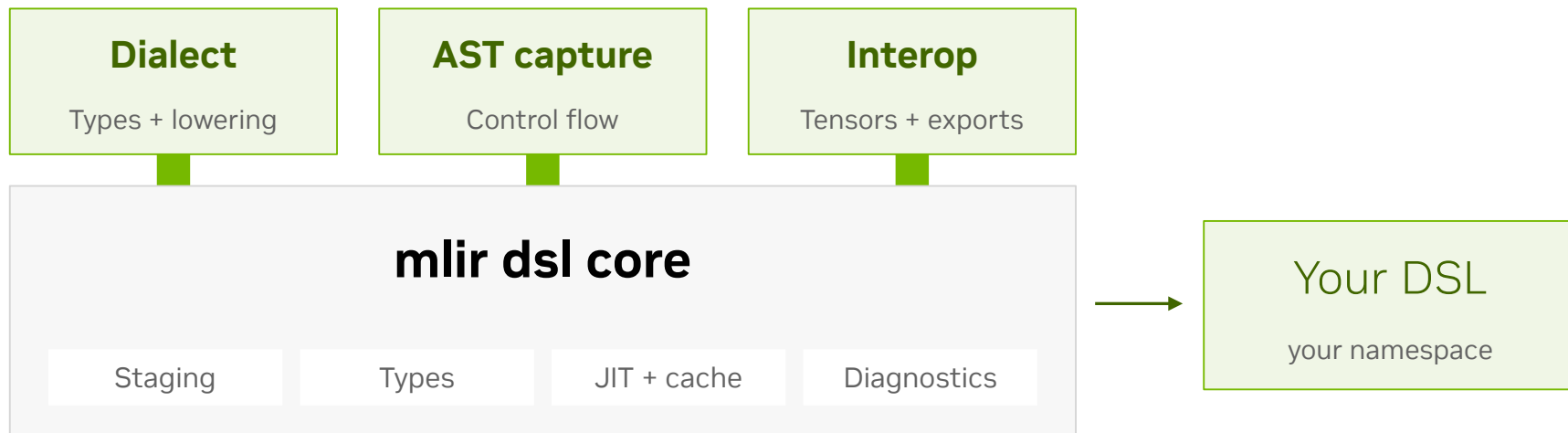
Your DSL selects it.

The core calls its hooks.

Example: an AST plugin captures Python control flow.

Build your DSL from the shared core

Choose the plugins your DSL needs.



Design principles

01 No new language.

Objects · imports · libraries

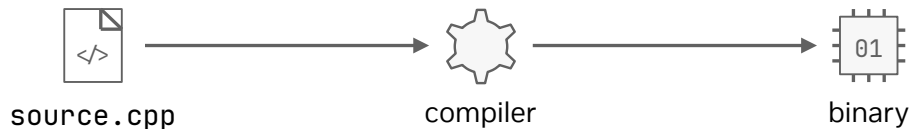
02 Python stays Python.

MLIR through a DSL namespace

This leads to **multi-stage programming**.

What is multi-stage programming?

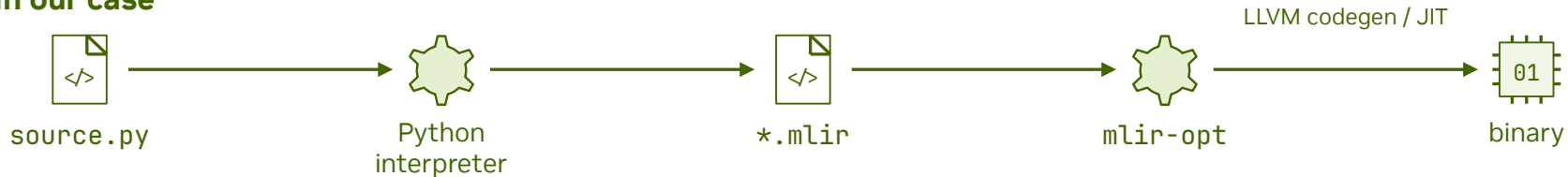
Traditional compiler



Multi-stage compilers



In our case



Why multi-stage programming?

01 METAPROGRAM

All of Python.

Objects · imports · libraries

02 DSL NAMESPACE

Explicit capture.

Types · operators · builders

```
import mlir.mlir_dsl as m
```

```
range(4)
```



Python loop

Runs now

```
m.range(4)
```



scf.for

Runs later

How to build a Python DSL for MLIR

Python

Python

```
import mlir.mlir_dsl as m

@m.jit
def scale(x: m.Int32, factor):
    return x * factor

scale(4, 3)
```

Objects · imports · metaprogramming

Trace: execute Python

Meta

factor = 3
Run Python

Staged

x = SSA
Emit MLIR

MLIR

Generated MLIR

```
// x is a runtime i32
%c3 = arith.constant 3 : i32
%r = arith.muli %x, %c3 : i32
func.return %r : i32
```

Compile

Test

Consume IR

One program. Two binding times.

Python / Meta

RUNS NOW

Objects · functions · dictionaries

DSL / Staged



compile

RUNS LATER

Typed values · MLIR operations

Agenda

- 01 Core + plugin architecture
- 02 Types + metaprogramming
- 03 Compile + control-flow plugins
- 04 Amir: mutation tracking
- 05 Tile IR: compose a new target

Build your language from BaseDSL

mydsl/__init__.py · setup excerpt

```
class MyDSL(BaseDSL):
    plugins = [ScfASTPreprocessorPlugin()]
    _jit_arg_adapter_scope = "mlir"

    def __init__(self):
        super().__init__(
            name="MY_DSL",
            dsl_package_name=["mydsl"],
            pass_sm_arch_name="cubin-chip",
            preprocess=True)
```

Choose

Plugins + namespace

Inherit

Staging · JIT · cache · env

Core API $\neq$ language namespace

For DSL authors

```
from mlir.dsl import (  
    BaseDSL, Plugin, DialectPlugin,  
    ASTPreprocessorPlugin, OpEmitter,  
    Int32, Pointer, register_leaf,  
)
```

For users of the reference DSL

```
import mlir.mlir_dsl as m  
  
@m.jit  
def scale(x: m.Int32, factor):  
    return x * factor  
  
scale(4, 3) # 12
```

Three plugin families

Dialect

`DialectPlugin`

Types + operations

Entries + lowering

`LLVM · SCF · GPU`

Python syntax

`ASTPreprocessorPlugin`

AST rewrite

Region callbacks

`SCF AST · custom AST`

Integration

`Plugin`

Argument adapters

ABI exports

`DLPack · PyTorch · TVM-FFI`

Extend a dialect—or replace its IR

Downstream plugin · implementation excerpt

```
class MyDialectPlugin(m.DialectPlugin):
    name = "mydialect"
    def register_dialects(self, context):
        mydialect.register_dialect(context)
    def pipeline_passes(self):
        return ["convert-mydialect-to-llvm"]

class MySubDSL(m.MlirDSL):
    plugins = [MyDialectPlugin()]
```

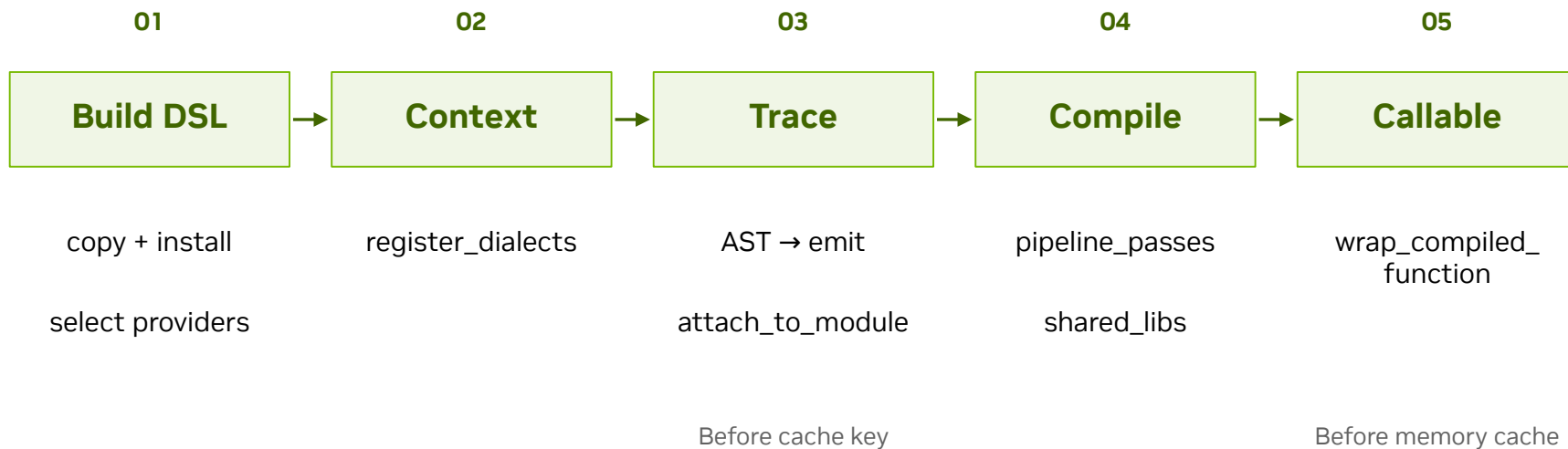
Add dialect + passes

Keep the default emitter

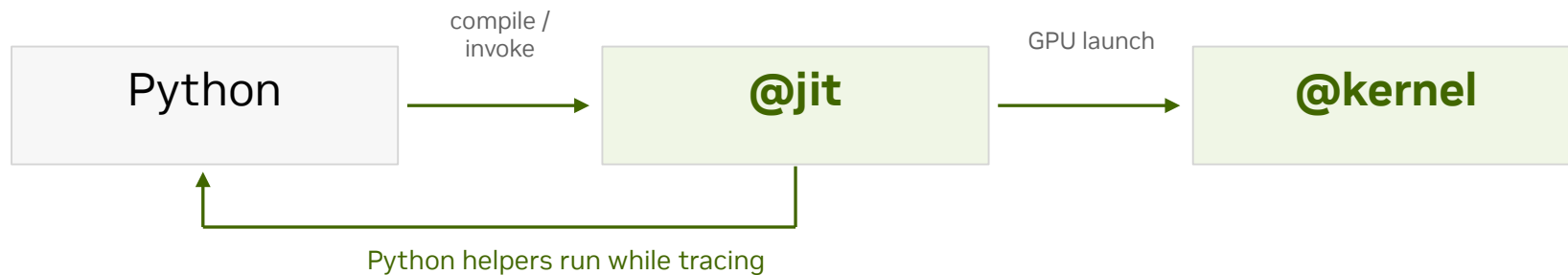
Supply an emitter

Replace SCF + LLVM defaults

A plugin follows the compilation lifecycle



@jit and @kernel



Host

Specialize · compile · call

GPU

Build kernel + launch

Direct launch

Python → @kernel
Target policy

Types keep policy; emitters choose IR

Bool · i1

Int2-128 / UInt8-128

FP64/32/16 · BF16

FP8/6/4 · TF32

Python or SSA value $\longrightarrow$ dtype + promotion $\longrightarrow$ active OpEmitter

Conversion

Python

MLIR

Capture

`m.Int32(7) → staged use`

`arith.constant : i32`

Promote / upcast

`x32 + y64`

`extsi → addi : i64`

Downcast

`y64.to(m.Int32)`

`arith.trunci`

Int → float

`x32.to(m.Float32)`

`arith.sitofp`

Float → int

`f32.to(m.Int32)`

`arith.fptosi`

Error

`x32.to(int)`

Runtime ≠ Python value

Shown: the LLVM emitter

One emitter, three value families

Core value	Dialect hooks
Scalar	mlir_type · const · add · cast
Vector	vector_type · broadcast · reduce
Pointer	pointer_type · load · store · ptr_add
Struct / frozen record	Flatten fields → rebuild record
Custom leaf	register_leaf

Structs need no dialect-specific aggregate type.

Local type inference

Annotations + local inference

```
@m.jit
def affine(x: m.Int32, factor):
    return x * factor + 1

affine(4, 3)

# x          → runtime i32
# factor     → known Python int
# result     → i32
```

Boundary

Annotation → runtime type

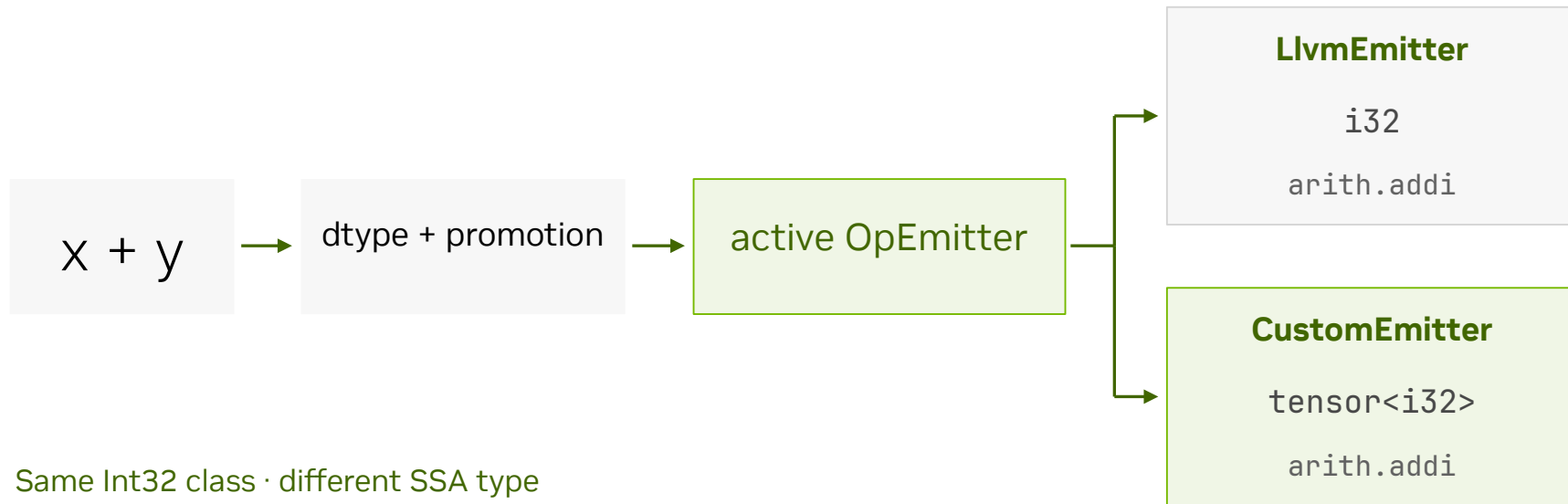
Operator

Promote → emit → wrap

Join

Same structure + types

Same Python types. Different IR.



Install the emitter as a dialect plugin

Rank-zero tensor plugin · excerpt

```
class TensorDialect(m.DialectPlugin):
    emitter = RankZeroTensorEmitter()
    host_gen_helper = m.LlvmHostGenHelper

class TensorDSL(m.MlirDSL):
    plugins = [TensorDialect()]

@TensorDSL.jit
def add6(x: m.Int32):
    return m.Int32(6) + x
```

Same expression · two traced forms

```
// Default LLVM world
%c6 = arith.constant 6 : i32
%r = arith.addi %c6, %x : i32

// Rank-zero tensor emitter
%c6 = arith.constant dense<6>
      : tensor<i32>
%r = arith.addi %c6, %x
      : tensor<i32>
```

Add mydialect.Tensor

Register a Tensor leaf

```
class Tensor:
    def __init__(self, value):
        self.value = value

m.register_leaf(
    Tensor,
    ir_types=lambda t: [t.value.type],
    ir_values=lambda t: [t.value],
    from_ir_values=lambda old, values:
        Tensor(values[0]))
```

Value protocol

Flatten / rebuild

Runtime boundary

Argument adapter

Python values specialize the program

Python

```
@m.jit
def affine(x: m.Int32, scale, bias):
    return x * scale + bias

affine(7, 3, 4) # 25
```

Generated MLIR

```
%c3 = arith.constant 3 : i32
%c4 = arith.constant 4 : i32
%a = arith.muli %x, %c3 : i32
%r = arith.addi %a, %c4 : i32
func.return %r : i32
```

scale, bias: Meta · x: Staged

Objects + dictionaries → arithmetic

Python polymorphism

```
class Op:
    def apply(self, x):
        raise NotImplementedError

class AddN(Op):
    def __init__(self, n): self.n = n
    def apply(self, x): return x + self.n

class MulN(Op):
    def __init__(self, k): self.k = k
    def apply(self, x): return x * self.k
```

Metaprogram

```
@m.jit
def pipeline(x: m.Int32, ops):
    for op in ops.values():
        x = op.apply(x)
    return x

ops = {"shift": AddN(2), "scale": MulN(3)}
pipeline(4, ops) # 18

# Captured: (x + 2) * 3
```

No runtime dictionary or virtual dispatch.

A GPU plugin supplies kernel + launch

GPU body · Pointer interface

```
@m.kernel
def fused(arr: m.Pointer[m.Float32],
         ops):
    tx, _, _ = m.thread_idx()
    x = arr[tx]
    for op in ops.values():
        x = op.apply(x)
    arr[tx] = x

# From @m.jit, for four elements:
# fused(arr, ops).launch(grid=[1], block=[4])
```

Core

Meta recipe · typed values

GPU plugin

gpu.func · gpu.launch_func

DLPack is a boundary plugin



Contiguous buffers · supported dtype + device

Compile now. Call later.

Current compile API

```
@m.jit
def axpb(x: m.Float32,
        a: m.Float32, b: int):
    return a * x + b

compiled = m.compile(axpb, 2.0, 3.0, 1)
compiled(2.0, 3.0, 1)  # 7.0
```

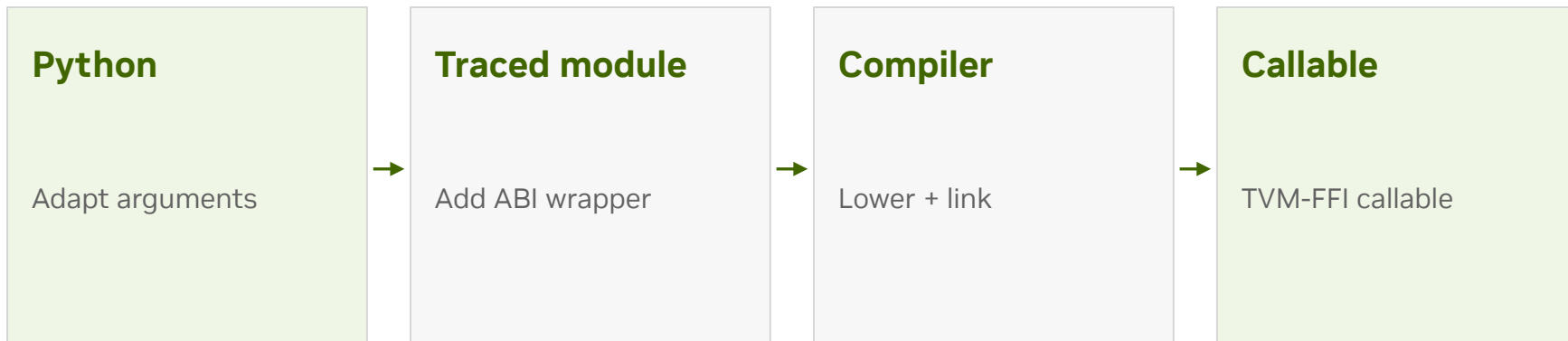
Signature

Runtime types + Meta b

Compiler provider

Pass manager + engine

An export plugin wraps both sides



Optional: `TvmFfiPlugin · MLIR_DSL_ENABLE_TVM_FFI=1`

Errors + MLIR remarks

Source error + fix

```
@m.jit
def expects_int(x: m.Int32):
    return x

expects_int("oops")
# ARG_ANNOTATION_MISMATCH
# Expected: Int32
# Pass an integer, or change the annotation.
```

MLIR_DSL_DEBUGINFO=1 · MLIR_DSL_REMARKS=.*

Errors

File · line · fix

Remarks

Trace + passes → records

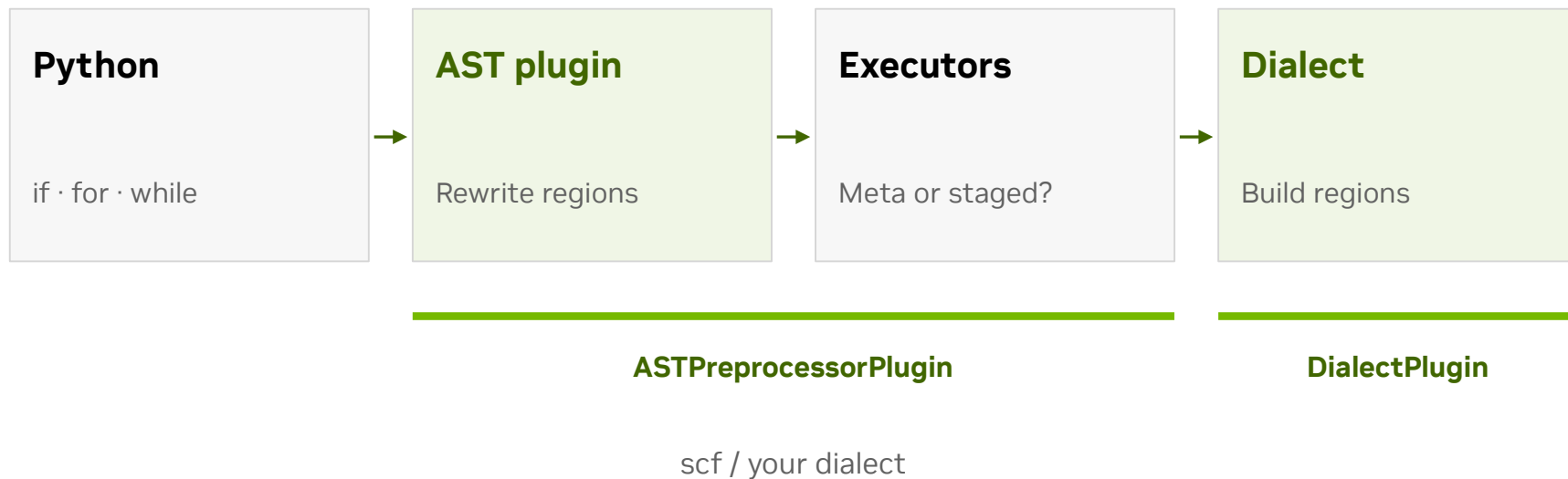
See what the compiler does

Tool	Setting / output
Source locations	Python line → MLIR location
Verify trace	VERIFY_TRACE
Remarks	REMARKS · REMARKS_OUTPUT
Inspect IR	PRINT_IR · KEEP_IR
Profile	PROFILE_COMPILER

Cache compilation work

Cache	Stores	Hit
Memory	Callable + engine	Reuse callable
Disk	Lowered bytecode	Skip passes
Identity	IR + settings + toolchain	Compatible programs

AST capture is a plugin, too



Syntax capture and IR emission are separate choices.

Assigned name → iter_arg

Python · find the carry

```
acc = m.Int32(0)
for i in range(n):
    acc = acc + i

# Defined outside: acc
# Written inside:  acc
# write_args = ["acc"]
```

MLIR · thread the SSA value

```
%sum = scf.for %i = %c0 to %n
    step %c1 iter_args(%acc = %zero)
    → (i32) : i32 {
    %next = arith.addi %acc, %i : i32
    scf.yield %next : i32
}
```

Flatten → build region → yield → rebuild

Records flatten into their fields

Dialect-neutral record

```
@m.struct
class Vec2:
    x: m.Float32
    y: m.Float32

@m.jit
def norm2(v: Vec2):
    return v.x*v.x + v.y*v.y
```

Two fields → two SSA arguments

```
func.func @norm2(
    %x: f32, %y: f32) → f32 {
    %a = arith.mulf %x, %x : f32
    %b = arith.mulf %y, %y : f32
    %r = arith.addf %a, %b : f32
    func.return %r : f32
}

// norm2(Vec2(x=3, y=4)) → 25
```

Frozen record · no aggregate SSA type

Swap the region callbacks

AST plugin · downstream sketch

```
class MyAST(ASTPreprocessorPlugin):  
    preprocessor_class = DSLPreprocessor  
  
    def executors(self, dsl):  
        return mydialect_executors  
  
class MyDSL(BaseDSL):  
    plugins = [MyAST(), MyDialect()]
```

AST plugin

Rewrite + callback map

Dialect plugin

Types · regions · lowering

Or build regions explicitly

Explicit control flow

```
@m.jit(preprocess=False)
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i, current, result in m.for_(
        0, n, 1, [acc]):
        m.yield_([current + i])
    return result
```

Builders

for_ · if_ · while_ · yield_

Boolean helpers

and_ · or_ · not_
any_ · all_

Python → MLIR → 45

Python · executed

```
@m.jit
def sum_upto(n: m.Int32):
    acc = m.Int32(0)
    for i in range(n):
        acc += i
    return acc

sum_upto(10)  # 45
```

MLIR · loop excerpt

```
%sum = scf.for %i = %c0 to %n
      step %c1 iter_args(%a = %zero)
      → (i32) : i32 {
    %next = arith.addi %a, %i : i32
    scf.yield %next : i32
  }
func.return %sum : i32
```

Fuse a Python recipe

Python · executed

```
@m.jit
def fused(arr: m.Pointer[m.Float32],
         ops):
    for i in m.range(4):
        x = arr[i]
        for op in ops.values():
            x = op.apply(x)
        arr[i] = x

# [0, 1, 2, 3] → [6, 9, 12, 15]
```

MLIR · loop body

```
%p = llvm.getelementptr %arr[%i]
      : (!llvm.ptr, i32) → !llvm.ptr, f32
%x = llvm.load %p : !llvm.ptr → f32
%a = arith.addf %x, %two : f32
%r = arith.mulf %a, %three : f32
llvm.store %r, %p : f32, !llvm.ptr
scf.yield
```

Staged array loop · Meta recipe loop

Richer mutation

Objects · aliasing · staged control flow

— Guray Ozen

Advanced mutation tracking

PyIR · Amir Tavakkoli

— Amir Tavakkoli

Tile IR: compose a target

Port to the new core · design sketch

```
class TileDSL(BaseDSL):  
    plugins = [  
        TileDialect(),  
        TileAST(),  
    ]  
  
# Package exports:  
jit = TileDSL.jit  
kernel = TileDSL.kernel
```

Reuse

Types · staging · cache · env

Supply

Emitter · AST · compiler

Tile IR: one emitter for its values

Core value	Tile representation	Emitter hooks
Int32	Rank-0 tile	mlir_type · add · cast
Vector	Rank-1 tile	vector_type · reduce
Pointer	Tile of pointers	pointer_type · load · store
Struct	Flattened fields	No aggregate hook

Direct types stay the same; the emitted IR changes.

Tile IR: a typed Python kernel

mlir dsl interface · target sketch

```
import mytile as tile

@tile.kernel
def sum_upto(n: tile.Int32):
    acc = tile.constant(0)
    for i in range(n):
        acc += i
    tile.print_tko("sum=%i\n", [acc])
```

Tile IR · loop body

```
%sum = for %i in (%zero to %n,
    step %one) : tile<i32>
    iter_values(%acc = %zero)
    → (tile<i32>) {
    %next = addi %acc, %i : tile<i32>
    continue %next : tile<i32>
}
```

Tile IR: entries + compiler are plugins

Dialect plugin · port sketch

```
class TileDialect(DialectPlugin):  
    emitter = TileEmitter()  
    host_gen_helper = TileHostEntry  
    kernel_gen_helper = TileKernelEntry  
    compiler_provider = tile_compiler  
  
    def pipeline_passes(self):  
        return tile_lowering_passes
```

Entry helpers

Arguments · return · launch

Compiler

Tile bytecode → GPU binary

Tile IR: target-owned launch

CUDA Python launch · backend excerpt

```
module = cuda_helpers.load_cubin_module(cubin)
kernel = cuda_helpers.get_kernel_function(
    module, name)
cuda_helpers.launch_kernel(
    kernel, num_tiles, (1, 1, 1), stream,
    smem_size=0, kernel_args=packed_args)
```

Desired Python call

```
sum_upto(10,
        num_tiles=[1, 1, 1])
```

Target provider

Compile · pack · launch

Choose plugins. Reuse the language core.